

## Desafío técnico

### Ingeniero de Desarrollo — Trycore Colombia

---

Hola,

Gracias por tu interés en hacer parte de Trycore. Este documento describe la prueba técnica que queremos que desarrolles como parte del proceso de selección.

Antes de entrar en detalle, queremos ser directos sobre lo que buscamos: no nos interesa si memorizas patrones de diseño ni si resuelves acertijos algorítmicos en tiempo récord. Nos interesa cómo piensas cuando te enfrentas a un problema que no conoces, cómo tomas decisiones cuando hay varias opciones válidas, y cómo construyes software que otra persona pueda entender y mantener.

Tienes cinco días calendario para entregar. Si tienes alguna duda, escríbenos — responder preguntas bien formuladas también nos dice mucho de cómo trabajas.

---

### El problema

Queremos construir una herramienta interna para que los líderes de proyecto puedan registrar el avance de sus actividades y entender, en tiempo real, si su proyecto va bien o mal en términos de cronograma y presupuesto.

La metodología que usaremos para ese análisis es el **Valor Ganado** (Earned Value Management), un estándar del PMI que probablemente no conoces. Eso está bien — de hecho, es parte intencional del ejercicio. Tendrás que aprenderlo durante el desarrollo.

La idea central del Valor Ganado es sencilla: no basta con saber cuánto has gastado ni cuánto has avanzado por separado. Lo que importa es la relación entre los dos. Un proyecto puede haber gastado el 60% del presupuesto habiendo completado solo el 40% del trabajo — y eso es una señal de alerta. Los indicadores EVM te permiten cuantificar exactamente eso.

---

### Qué debes construir

Una aplicación fullstack que permita gestionar proyectos y sus actividades, y que calcule automáticamente los indicadores de Valor Ganado.



Bogotá COL



Santo Domingo Dom., RD



San José CR

## Backend

Necesitamos una API REST que exponga operaciones para crear, editar y eliminar proyectos y actividades. Cada actividad debe registrar los siguientes datos:

- Nombre
- Presupuesto total planificado (BAC – Budget at Completion)
- Porcentaje de avance planificado a la fecha de corte
- Porcentaje de avance real completado
- Costo real incurrido hasta la fecha (AC – Actual Cost)

Con esos datos, el sistema debe calcular automáticamente los siguientes indicadores por actividad y de forma consolidada por proyecto:

| Indicador                        | Fórmula             |
|----------------------------------|---------------------|
| PV – Planned Value               | % planificado × BAC |
| EV – Earned Value                | % completado × BAC  |
| CV – Cost Variance               | EV – AC             |
| SV – Schedule Variance           | EV – PV             |
| CPI – Cost Performance Index     | EV / AC             |
| SPI – Schedule Performance Index | EV / PV             |
| EAC – Estimate at Completion     | BAC / CPI           |
| VAC – Variance at Completion     | BAC – EAC           |

El API también debe retornar la interpretación de CPI y SPI: si el proyecto está bajo presupuesto o sobre presupuesto, adelantado o atrasado. Un CPI mayor a 1 indica eficiencia en costos; menor a 1 indica que se está gastando más de lo que se avanza. El SPI funciona con la misma lógica pero sobre el cronograma.

## Frontend

Un dashboard donde el líder de proyecto pueda ingresar y editar sus actividades, y ver el resultado del análisis en tiempo real. Debe incluir la tabla de actividades con sus indicadores calculados, los indicadores consolidados del proyecto, una indicación visual del estado de CPI y SPI, y una gráfica que compare PV, EV y AC por actividad.

No pedimos un diseño elaborado. Pedimos que la información sea clara y que quien la mire entienda de un vistazo si el proyecto va bien o mal.

---

## Estándares que debe cumplir el desarrollo

**Pruebas unitarias.** Toda la lógica de cálculo EVM debe estar cubierta con pruebas unitarias. Esto incluye los casos borde: qué pasa cuando AC es cero, cuando no hay actividades, cuando el avance real es cero. Esperamos una cobertura mínima del 80% sobre la capa de negocio. Cada endpoint debe tener al menos un test de integración que valide el contrato de respuesta.

**Cero code smells.** El código debe estar limpio. Sin bloques comentados, sin variables sin usar, sin números o strings mágicos dispersos por el código. Los nombres de variables, métodos y clases deben ser descriptivos. La lógica de negocio no debe vivir en los controladores. Si una función hace más de una cosa, probablemente deba dividirse. Si un bloque de lógica se repite más de dos veces, debe abstraerse. Recomendamos configurar un linter en el proyecto – si lo haces, incluye la configuración en el repositorio.

**Gitflow estricto.** El historial de tu repositorio es parte de la entrega. La estructura de ramas debe seguir el flujo estándar: `main` para producción, `develop` como rama de integración, ramas `feature/*` por cada funcionalidad, y al menos una rama `release/*` antes del merge final a `main`. Cada feature debe integrarse a `develop` mediante un Pull Request, aunque trabajes solo. Los mensajes de commit deben ser descriptivos y en imperativo: `Add EVM calculation service`, `Fix CPI edge case when AC is zero`. Mensajes como `fix`, `cambios` o `wip` no son aceptables.

**OpenAPI/Swagger.** Valoramos positivamente que el API esté documentado con la especificación OpenAPI. Si lo implementas, debe ser accesible localmente en `/api-docs` o `/swagger-ui`, y cada endpoint debe incluir descripción, esquemas de request y response, y los posibles códigos de error. Hacerlo bien demuestra que entiendes el contrato del API como un artefacto de comunicación, no solo como documentación.

---



Bogotá COL



Santo Domingo Dom., RD



San José CR

## Stack tecnológico

Usa el stack con el que tengas mayor dominio. Nuestra preferencia es Java con Spring Boot o Python con FastAPI en el backend, base de datos relacional (PostgreSQL idealmente), y Angular o React en el frontend. Si eliges algo diferente, explica por qué en tu documento de proceso.

---

## Los tres entregables

### 1. El repositorio

En GitHub o GitLab — no aceptamos archivos comprimidos porque necesitamos ver el historial de commits. Debe incluir un [README.md](#) con instrucciones para correr el proyecto localmente y el script de inicialización de la base de datos.

### 2. El documento **AI\_PROCESS.md**

Este documento es tan importante para nosotros como el código. Debe estar en el repositorio e incluir lo siguiente:

- Las herramientas de IA que usaste y por qué elegiste esas.
- Todos los prompts que enviaste, copiados textualmente y en orden cronológico — no los resumas ni los parafrasees.
- Cómo aprendiste EVM: qué le preguntaste a la IA, cómo validaste que entendiste las fórmulas antes de implementarlas.
- Dos decisiones donde no seguiste lo que la IA te sugirió, explicando qué propuso y por qué tomaste un camino diferente. Cómo verificaste que los cálculos son correctos — no solo que el código funciona, sino que los números tienen sentido.
- Una decisión de arquitectura que tomaste de forma independiente.
- Una reflexión honesta sobre qué harías diferente si repitieras el ejercicio.

**No esperamos un documento perfecto. Esperamos uno honesto.**

### 3. El video

- Máximo diez minutos, grabando tu pantalla. Sin edición elaborada. Queremos **escucharte y verte** explicar, en tus propias palabras, qué es el Valor Ganado y cómo funciona — como si se lo explicararas a un colega que nunca lo ha escuchado.
- Luego muéstranos la arquitectura de tu solución, una decisión técnica que te resultó difícil, una demo del sistema funcionando con al menos un proyecto y tres actividades, y cómo se ve tu flujo de trabajo con IA en el documento de proceso.



Bogotá COL



Santo Domingo Dom., RD



San José CR

La fluidez al explicar algo que aprendiste durante el ejercicio nos dice más que cualquier algoritmo que hayas memorizado.

---

## Cómo evaluamos

El mayor peso de la evaluación está en el video y en el documento de proceso, no en el código. Un código impecable producido sin comprensión real vale menos para nosotros que un código más modesto respaldado por razonamiento claro.

Lo que buscamos ver es un ingeniero que use la IA para pensar mejor, no para evitar pensar. Que sepa cuándo la IA tiene razón y cuándo no. Que escriba código que otro pueda leer y mantener. Que entienda lo que construyó lo suficientemente bien como para explicarlo sin leer.

Lo que no queremos ver es un documento de proceso genérico escrito después del hecho, un video donde el candidato lee en lugar de explicar, o pruebas unitarias que solo verifican que las funciones retornan algo.

---

Queda pendiente que nos confirmes la recepción de este documento. Cualquier pregunta, escríbenos.

Éxitos,

**Equipo de Tecnología**

Trycore Colombia